

AeroPico FC v0.2.0

Professional Software Architecture Review

Fixed-Wing Flight Controller Firmware
Raspberry Pi Pico 2 / RP2350 Target

Hazırlayan: Claude Sonnet
Tarih: Haziran 2026

BÖLÜM 1 — Executive Summary

AeroPico FC v0.2.0, sabit kanatlı İHA'lar (UAV) için Raspberry Pi Pico 2 (RP2350) üzerinde çalışan açık kaynaklı bir uçuş kontrolcüsü yazılımıdır. Toplam yaklaşık 2.200 satır C++ kodundan oluşan bu proje; çift çekirdekli mimari, FreeRTOS görev izolasyonu, Madgwick filtrelemesi ve MAVLink desteği gibi profesyonel düzey teknolojileri başarıyla bir araya getirmektedir.

1.1 Genel Değerlendirme

Proje, bir öğrenci/hobi projesinin çok ötesinde; gerçek bir gömülü sistem mühendisliği bilgisi ve disiplini sergilemektedir. Özellikle RP2350 donanımının PIO, DMA ve çift çekirdek yeteneklerinin doğrudan SDK üzerinden kullanılması, PX4 ve ArduPilot gibi olgun yazılımların temel mimari kararlarıyla örtüşmektedir.

1.2 Güçlü Yönler

- PIO tabanlı jitter-free servo PWM çıkışı — donanım hızlandırmalı, CPU'yu meşgul etmeyen hassas timing.
- DMA + ping-pong buffer ile MPU6050 sensör okuma — ISR veya polling yerine gerçek zamanlı DMA mimarisi.
- Madgwick filtresi (9-DOF, kuaternion tabanlı) ile sıcaklık kompanzasyonu eklentisi.
- Cascaded PID: Açık dış döngüsü → Rate iç döngüsü mimarisi, profesyonel autopilot standardı.
- Lock-free SPSC Ring Buffer — core0/core1 arası veri paylaşımı için mutex yerine atomic DMB bariyeri.
- FreeRTOS core affinity ile görev izolasyonu — SensorTask/Core0, Control döngüsü/Core1.
- MAVLink v2 parser + sender, PIO UART üzerinden — GPS ve RC portları serbest kalıyor.
- SBUS protokol desteği (Futaba/FrSky uyumlu), hardware failsafe bit + timeout logic.
- BootLogger ile yapılandırılmış başlatma raporu ve WHO_AM_I sensör doğrulaması.
- ParamManager ile MAVLink PARAM_SET/REQUEST protokolü — GCS üzerinden PID tuning altyapısı.

1.3 Kritik Eksikler ve Riskler

⚠ Watchdog timer config.h'da tanımlanmış ancak ana döngüde hiçbir yerde başlatılmamış ve beslenmiyor. Bu durum, watchdog korumasının tamamen işlevsiz olduğu anlamına gelir.

⚠ FreeRTOS task stack büyüklükleri (2048 word) mevcut telemetry + MAVLink yüküyle yetersiz kalabilir. Stack overflow detection mekanizması eksik.

⚠ Sensor fusion'daki updateIMU() fonksiyonunda Madgwick gradient hesabı eksik — sadece gyro integrasyonu yapılıyor. Aksel katkısı olmadığı için ivmelenme sırasında drift oluşur.

⚠ IMU.cpp (legacy complementary filter) ve SensorFusion.cpp (Madgwick) paralel var. IMU.cpp hiç kullanılmıyor; ancak bu çift başlık dosyası karışıklığa yol açabilir.

⚠ PID anti-windup sadece integral'i ± 100 ile sınırlıyor. Output limiting ve back-calculation anti-windup eksik. Yüksek hata durumlarında integral windup problemi devam eder.

1.4 Profesyonellik Puanı (Genel)

Kriter	Puan	Not
Mimari Tasarım	8/10	Çift çekirdek + FreeRTOS izolasyonu mükemmel
Donanım Kullanımı	9/10	PIO, DMA, raw I2C — RP2350 tam kapasitede
Gerçek Zamanlı Uygunluk	7/10	500Hz döngü, ancak watchdog aktif değil
Sensör Katmanı	7/10	WHO_AM_I doğrulaması var, kalibrasyon yok
Sensor Fusion	6/10	updateIMU() gradient eksik, drift riski
Kontrol Algoritmaları	7/10	Cascaded PID iyi, anti-windup yetersiz
Güvenlik / Failsafe	6/10	Watchdog aktif değil, stack guard yok
Haberleşme	8/10	MAVLink v2, Blackbox, PIO UART — sağlam
Kod Kalitesi	8/10	Okunabilir, modüler, iyi yorumlanmış
Ticari Potansiyel	7/10	v1.0 ile ciddi prototip seviyesine ulaşır

1.5 Ticari Potansiyel

Mevcut haliyle akademik araştırma, hobi prototipi ve firmware öğrenme platformu olarak yüksek değer taşımaktadır. Watchdog aktifleştirilmesi, EKF entegrasyonu ve GPS navigasyon eklenmesi halinde küçük ölçekli ticari UAV projelerinde kullanılabilir potansiyele sahiptir. PX4/ArduPilot alternatifi olarak konumlandırılabilmesi için en az v1.0 hedeflerine ulaşılması gerekmektedir.

BÖLÜM 2 — Yazılım Mimarisi

2.1 Katmanlı Mimari Analizi

AeroPico FC, belirgin katmanlar halinde organize edilmiş bir mimari sergilemektedir:

- Donanım Katmanı: PIO state machine'ler (PWM, UART), DMA kanalları, raw I2C — SDK doğrudan kullanımı.
- Sürücü Katmanı (src/drivers/): Sensors, RX, Output, PioUart — HAL benzeri soyutlama.
- Çekirdek Katmanı (src/core/): FlightManager, SensorFusion, PID, Mixer — uçuş mantığı.
- Telemetri Katmanı (src/telemetry/): MAVLink, Blackbox, ParamManager — dış haberleşme.
- Yardımcı Katman (src/Utils/): Logger, BootLogger, MathUtils — cross-cutting concerns.

2.2 Dosya Organizasyonu ve Modüler Yapı

Modül	Dosya	Satır	Sorumluluk	Kalite
FlightManager	FlightManager.cpp/h	~110	Çekirdek veri akışı + arm/disarm	✓
SensorFusion	SensorFusion.cpp/h	~190	Madgwick 9-DOF quaternion	✓
SensorManager	Sensors.cpp/h	~210	MPU6050 DMA + GY87	✓
PID	PID.cpp/h	~40	Cascaded PID kontrolcüsü	△
FixedWingMixer	FixedWingMixer.cpp/h	~80	Servo mixing + gain	✓
Output	Output.cpp/h	~60	PIO PWM servo çıkışı	✓
RXManager	RX.cpp/h	~80	SBUS parser + failsafe	✓
MavlinkHandler	MavlinkHandler.cpp/h	~160	MAVLink v2 parse + send	✓
Blackbox	Blackbox.cpp/h	~50	Uçuş verisi loglama	△
ParamManager	ParamManager.cpp/h	~100	GCS parametre protokolü	✓
RingBuffer	RingBuffer.h	~70	Lock-free SPSC buffer	✓
IMUManager	IMU.cpp/h	~50	Legacy, KULLANILMIYOR	✗

2.3 Modüller Arası Bağımlılıklar

main.cpp → FlightManager → SensorManager + RXManager + SensorFusion (Core 0 tarafında)

main.cpp → SystemTimer::init() → multicore_launch_core1() → PID döngüsü + FixedWingMixer (Core 1 tarafında)

MavlinkHandler → FlightManager (extern referans) — bu bağımlılık header-level ve circular dependency riski taşıyor.

Mimari Not

FlightManager'ın extern olarak header'da declare edilmesi ve MavlinkHandler'ın onu doğrudan referans alması, katmanlar arası sıkı bağlantı (tight coupling) oluşturuyor. İdeal çözüm: FlightData'yı bir callback veya shared state interface üzerinden iletmek.

2.4 SOLID Prensipleri Analizi

- Single Responsibility: Her sınıf tek bir işe odaklanıyor. Çok iyi uyum.
- Open/Closed: MixerSettings struct üzerinden genişletilebilirlik mevcut. SensorFusion IMU/9DOF modüler. İyi.
- Liskov Substitution: Kalıtım kullanılmıyor, prensip ihlali yok.
- Interface Segregation: Header dosyaları minimal ve odaklı. İyi.
- Dependency Inversion: MavlinkHandler → FlightManager doğrudan bağımlılık, DI ihlali. Geliştirme gerekli.

2.5 Ölçeklenebilirlik Değerlendirmesi

Mevcut mimari, 4 servo çıkışı + 1 motor + sensör döngüsü için yeterli. Çoklu uçuş modu (MANUAL/FBWA/CRUISE), baro-tabanlı irtifa tutma, GPS waypoint navigasyonu eklenmesi halinde FlightManager'ın sorumlulukları aşılacaktır. v0.4+ için FlightManager'ın ayrı FlightMode, AltitudeController ve NavigationController sınıflarına bölünmesi önerilir.

BÖLÜM 3 — Gerçek Zamanlı Sistem Analizi

3.1 FreeRTOS Görev Mimarisi

Projede FreeRTOS, `multicore_launch_core1()` ile kombine edilmiştir. Bu hibrit yaklaşım; Core 0'da FreeRTOS scheduler, Core 1'de bare-metal döngü şeklinde çalışır.

Görev	Core	Öncelik	Periyot	Stack
SensorTask	Core 0	2 (yüksek)	2 ms (500 Hz)	2048 word
TelemetryTask	Core 0	1 (düşük)	20 ms (50 Hz)	2048 word
PID Döngüsü	Core 1	Bare-metal	2 ms (500 Hz)	N/A

3.2 Core Affinity ve Task Scheduling

`xTaskCreateAffinitySet()` ile `SensorTask` ve `TelemetryTask` her ikisi de Core 0'a sabitlenmiş (affinity mask = 0x01). Core 1 doğrudan `multicore_launch_core1()` ile başlatılıyor. Bu yaklaşım, determinizm açısından doğru bir tasarım kararıdır.

Analiz

`SensorTask` ve `TelemetryTask` ikisi de Core 0'da. `SensorTask` önceliği 2, `TelemetryTask` 1. 500Hz sensör döngüsü, 50Hz telemetri döngüsüne preempt edebilir. `vTaskDelayUntil()` doğru kullanılmış — saat kayması oluşmaz. Bu, profesyonel kalitede bir FreeRTOS kullanımıdır.

3.3 Mutex ve Senkronizasyon

`SensorManager` içinde `mutex_t _mutex` kullanılıyor. Bu Pico SDK'nın native mutex'i olup FreeRTOS'un `portMUXES` ile değil, hardware mutex ile çalışıyor.

⚠ `mutex_enter_blocking()` fonksiyonu FreeRTOS context'te çağrıldığında başka görevlerin çalışmasını engelleyebilir (busy-wait). FreeRTOS uyumlu `xSemaphoreTake()` kullanımı daha güvenli olurdu.

3.4 Lock-Free Ring Buffer (SPSC) Analizi

`RingBuffer<FlightData, 4>` implementasyonu, Single Producer Single Consumer için `__dmb()` (Data Memory Barrier) ile güçlendirilmiş lock-free bir tasarım sunmaktadır. Core 0 producer, Core 1 consumer. Bu pattern, gerçek RTOS tasarımlarında (FreeRTOS, Zephyr) sıklıkla kullanılan ve doğruluğu kanıtlanmış bir yaklaşımdır.

- `volatile _head` ve `_tail`: Compiler optimizasyonuna karşı korunmuş.
- `__dmb()` bariyeri: Yazma sırasını garantiliyor — ARM Cortex-M33 için doğru.
- `SIZE=4`: 500Hz üretim/tüketim hızında yeterli. En kötü senaryo: 4 slot * 2ms = 8ms tampon.

3.5 PIO State Machine Analizi

Projede `pio0` PWM servolar için, `pio1` UART için kullanılmaktadır. 4 PWM state machine (aileron, elevator, rudder, throttle) + 2 UART state machine (TX, RX) = 6 SM. RP2350'nin `pio0` ve `pio1`'i her biri 4 SM içerir, toplam 8 SM mevcuttur. 6 SM kullanımı limitlere yakın ama makul.

PIO	SM	Program	Pin
pio0	0	pwm_servo	GP16 (Aileron)
pio0	1	pwm_servo	GP17 (Elevator)
pio0	2	pwm_servo	GP18 (Rudder)
pio0	3	pwm_servo	GP19 (Throttle)
pio1	0	pio_uart_tx	GP12 (ESP TX)
pio1	1	pio_uart_rx	GP13 (ESP RX)

3.6 Worst Case Execution Time (WCET)

500Hz döngü = 2000µs bütçe. Aşağıdaki tahmini WCET değerleri, RP2350 @133MHz için hesaplanmıştır:

İşlem	Tahmini WCET	Not
DMA başlatma	~5µs	CPU bloke değil
Madgwick update()	~150µs	Float ops, FPU hızlandırılmış
Cascaded PID (3 eksen)	~20µs	Basit float aritmetik
FixedWingMixer	~10µs	map() + constrain()
MAVLink parse	~30µs	Byte-by-byte parser
Blackbox snprintf	~50µs	String format maliyetli
TOPLAM	~265µs	2000µs bütçenin %13'ü

WCET Sonucu

500Hz döngüde tahmini CPU kullanımı ~13%. Bu, çok iyi bir real-time margin anlamına gelir. GY87 mag/baro okuma (Wire.h blocking) en büyük gecikme kaynağı olabilir — bu Wire çağrılarını DMA yerine polling yapıyor.

BÖLÜM 4 — Sensör Katmanı Analizi

4.1 MPU6050 Sürücüsü — Raw I2C + DMA

MPU6050 sürücüsü, Arduino Wire kütüphanesi yerine doğrudan Pico SDK'nın `i2c_init()`, `i2c_write_blocking()` ve DMA API'sini kullanmaktadır. Bu, profesyonel bir gömülü sistem yazılımı için doğru tercih olup CPU'nun sensör verisini beklemesini ortadan kaldırmaktadır.

Yapılandırma Analizi

- Akselerometre: $\pm 8g$ (MPU6050_REG_ACCEL_CFG = 0x10) — Scale: 1/4096 g/LSB. Uçuş için uygun aralık.
- Jiroskop: $\pm 500^\circ/s$ (MPU6050_REG_GYRO_CFG = 0x08) — Scale: 1/65.5 $^\circ/s/LSB$. Sabit kanat için ideal.
- DLPF: 21Hz bant genişliği (CONFIG = 0x04) — Servo titreşimini filtreler, gecikme $\sim 13ms$. Kabul edilebilir.
- Sıcaklık okuması: Register 0x41, formül $T = raw/340 + 36.53^\circ C$ — Datasheet uyumlu.

DMA Implementasyon Analizi

DMA transfer akışı: `_mpu_start_dma_read()` → I2C register adresi yaz → I2C'ye 14 byte READ komutu gönder → DMA, I2C RX FIFO'dan doğrudan RAM'e aktar. CPU engellenmez.

⚠ `_mpu_start_dma_read()` içinde `i2c_write_blocking()` çağrısı hala blocking! Register adresi yazarken CPU bekliyor. Tam DMA pipeline için I2C TX da DMA'ya alınmalıdır. Bu küçük bir gecikme ($< 100\mu s$) ancak gerçek non-blocking için düzeltilmeli.

4.2 IIR Low-Pass Filter

Akselerometre verisi üzerinde IIR filtre uygulanmıştır: $y[n] = 0.15 * x[n] + 0.85 * y[n-1]$

Alpha = 0.15 ile 500Hz'de efektif kesim frekansı yaklaşık 13Hz. Bu, vibrasyon gürültüsünü iyi bastırır. Jiroskop filtresizdir — Madgwick zaten integrasyon yapıyor, doğru karar.

4.3 GY87 Modülü (HMC5883L + BMP085)

USE_GY87 makrosu ile koşullu derleme yapılmakta, magnetometre ve barometre Wire kütüphanesi üzerinden Adafruit driver'ları ile okunmaktadır.

⚠ BMP085 sürücüsü kullanılıyor ancak GY87 modülünde genellikle BMP180 bulunur. BMP085 ve BMP180 register uyumludur ama bazı hassasiyet farklılıkları olabilir. `config.h`'da net belirtilmeli.

⚠ `mag.getEvent()` ve `bmp.getEvent()` çağrıları Wire.h üzerinden blocking I2C yapıyor. Bu çağrılar, `SensorManager::update()` içinde DMA ile okunan MPU6050 sonrasına senkronize edilmiş ama Wire blocking kalıyor. En kötü senaryoda $\sim 3-5ms$ gecikme ekleyebilir.

4.4 Sensör Doğrulama

WHO_AM_I (register 0x75) kontrolü: MPU6050 için 0x68 bekleniyor, mismatch'te `BootLogger::fail()` çağrılıyor ve `_imuAvailable = false` yapılıyor. Bu, profesyonel bir defensive programming pratiğidir.

Sensör	Doğrulama	Hata İşleme	Kalibrasyon
MPU6050	WHO_AM_I = 0x68 ✓	fail() log + disable ✓	Yok ✗

HMC5883L	mag.begin() ✓	hasMag = false ✓	Yok ✗
BMP085	bmp.begin() ✓	hasBaro = false ✓	Yok ✗

4.5 Kalibrasyon Eksikliği

⚠ MPU6050 için accelerometer ve gyroscope bias kalibrasyonu yok. İlk boot'ta sensör offset'leri sıfır kabul ediliyor. Gerçek uçuşta drift ve offset hataları oluşacaktır. En azından statik bias hesaplama (N sample ortalaması) eklenmelidir.

BÖLÜM 5 — Sensor Fusion Analizi

5.1 Madgwick Filtresi Implementasyonu

SensorFusion.cpp'de Sebastian Madgwick'in orijinal algoritmasının tam implementasyonu mevcuttur. Kuaternion tabanlı attitude estimation, gyroscope integrasyonu ve gradient descent optimizasyonu içermektedir.

update() — 9-DOF (GY87 için)

Tam Madgwick implementasyonu: akselerometre + manyetometre ile gradient hesabı, kuaternion normalizasyonu, Euler açılarına dönüşüm. Beta parametresi = 0.08 (init'te). Bu, filtre kararlılığı ile drift düzeltme arasındaki trade-off'u iyi dengeler.

updateIMU() — 6-DOF (MPU6050 için)

⚠ updateIMU() fonksiyonu yalnızca gyro integrasyonu yapıyor! Madgwick'in 6-DOF versiyonunda bulunması gereken accelerometer-based gradient correction eksik. Bu, ivmelenme durumlarında yatay açılarda drift oluşmasına yol açar. update() fonksiyonunun 6-DOF sürümü eklenmeli.

5.2 Sıcaklık Kompanzasyonu

SensorFusion, _gyroTempCoeff = 0.004 ile jiroskop drift'ini sıcaklığa göre düzeltiyor. Referans sıcaklık 25°C. Bu değer MPU6050 datasheet ile uyumludur.

Değerlendirme

Sıcaklık kompanzasyonu eklenmesi, v0.2 için çok ileri bir özellik ve proje olgunluğunu gösteriyor. Ancak _gyroTempCoeff bireysel sensöre göre kalibre edilmeden sabit bir değer olarak kullanılıyor.

5.3 Kuaternion ve Euler Analizi

computeAngles() fonksiyonu kuaterniondan Euler açılarına (roll, pitch, yaw) dönüşüm yapar:

```
roll = atan2(2*(q0*q1 + q2*q3), 1 - 2*(q1^2 + q2^2))
pitch = asin(2*(q0*q2 - q3*q1))
yaw = atan2(2*(q0*q3 + q1*q2), 1 - 2*(q2^2 + q3^2))
```

Bu dönüşüm matematiksel olarak doğrudur. Gimbal lock, pitch = $\pm 90^\circ$ durumunda asin() fonksiyonunun ± 1 limitinde oluşur. Kuaternion tabanlı fusion bu riski minimize eder ama tam olarak ortadan kaldırmaz.

5.4 invSqrt() Fonksiyonu

SensorFusion.cpp'de invSqrt() tanımlı ancak implementasyonu $1.0f/\sqrt{x}$ ile yapılmış. Quake III'ün ünlü bit-manipulation trick'i kullanılmamış. RP2350'nin FPU'su ile donanım sqrt() zaten hızlı, bu yüzden pratik bir sorun değil.

5.5 Noise ve Drift Değerlendirmesi

Kaynak	Etki	Mevcut Çözüm	Eksik
--------	------	--------------	-------

Gyro drift	Yaw drift	Madgwick gradient	Mag kalibrasyon
Accel vibrasyon	Pitch/roll noise	IIR filter alpha=0.15	Notch filter
Mag distortion	Yaw hata	Madgwick mag fusion	Hard/soft iron comp.
Temp drift	Gyro bias	Sıcaklık comp. koeff	Bireysel kalibrasyon
Baro noise	Yükseklik	Yok	LPF + Kalman

BÖLÜM 6 — Kontrol Algoritmaları Analizi

6.1 PID Kontrolcüsü

PID.cpp'de standart PID implementasyonu mevcuttur. `__not_in_flash_func()` niteleyicisi ile RAM'de çalıştırılarak flash erişim gecikmesi önlenmiştir.

Mevcut Uygulama

```
P = kp * error
integral += error * dt; integral = constrain(integral, -100, 100);
D = kd * (error - prev_error) / dt
output = P + I + D
```

Anti-Windup Analizi

⚠ Mevcut anti-windup yalnızca integral toplamını ± 100 ile sınırlıyor (clamping). Bu yeterli değil. Çıkış satüre olduğunda bile integral büyümeye devam ediyor (windup). Back-calculation veya conditional integration anti-windup eklenmeli.

Önerilen Anti-Windup

if (abs(output) >= output_limit) { integral -= error * dt; } — Output sınırına ulaşıldığında integral'i geri al (back-calculation). Veya PX4 tarzı: `output = clamp(P + I + D, min, max); integral_clamp = clamp(integral, min/ki, max/ki);`

6.2 Cascaded PID Mimarisi

SystemTimer.cpp'de 5 adet PID nesnesi kullanılmaktadır:

- rollAnglePID / pitchAnglePID: Dış döngü (Angle → Rate setpoint)
- rollRatePID / pitchRatePID: İç döngü (Rate → Servo correction)
- yawRatePID: Yaw sadece rate kontrolü (sabit kanat için doğru)

Cascaded mimari, sabit kanatlı uçaklar için endüstri standardıdır. PX4 ve ArduPilot'un FBWA (Fly-By-Wire-A) modu ile aynı mantığı kullanmaktadır.

6.3 RC Giriş İşleme

RC kanalları 1000-2000 μ s aralığına normalize edilip `mapFloat()` ile fiziksel birimlere çevriliyor:

```
targetRoll    = mapFloat(aileron, 1000, 2000, -30°, +30°)
targetPitch   = mapFloat(elevator, 1000, 2000, -20°, +20°)
targetYawRate = mapFloat(rudder, 1000, 2000, -100°/s, +100°/s)
```

Bu değerler `config.h`'daki `MAX_ROLL_ANGLE`, `MAX_PITCH_ANGLE`, `MAX_YAW_RATE` sabitleriyle yönetilmektedir. Uçuş testine göre ayarlanabilir yapısı iyi.

6.4 Mixer Analizi

İki Mixer sınıfı mevcuttur: `Mixer.h` (Servo.h ile) ve `FixedWingMixer.h` (PIO Output ile). `main.cpp`'de `SystemTimer.cpp` üzerinden `FixedWingMixer` kullanılmaktadır. `Mixer.h` (Servo kütüphanesi versiyonu) kullanılmıyor.

⚠ Mixer.h içindeki Servo tabanlı sınıf ve FixedWingMixer aynı işlevi yapıyor. Kod çoğalması (duplication) var. Mixer.h silinmeli veya FixedWingMixer'a birleştirilmeli.

6.5 Feed-Forward Eksikliği

⚠ Feed-forward kanalı mevcut değil. Özellikle throttle değişimlerinde pitch düzeltmesi veya banking sırasında yaw koordinasyonu için feed-forward kritik öneme sahiptir. ArduPilot'un TECS sistemi bunu gelişmiş şekilde uygular.

BÖLÜM 7 — Güvenlik Analizi

7.1 Failsafe Mekanizması

RX.cpp'de çift katmanlı failsafe uygulanmıştır:

- SBUS protokol failsafe biti: sbus_data.failsafe == true olduğunda tetiklenir.
- Timeout failsafe: 500ms (FAILSAFE_TIMEOUT_MS) içinde geçerli sinyal gelmezse aktif olur.

Failsafe aktif olduğunda: throttle = PWM_MIN, aileron/elevator/rudder = PWM_NEUTRAL. Bu, güvenli bir davranıştır.

Değerlendirme

Failsafe implementasyonu sağlam ve çift katmanlı. SBUS protokolünün kendi güvenlik mekanizması + yazılım timeout ile redundant bir yapı oluşturuluyor. FlightManager da rx.isValid() kontrolü yapıyor. ✓

7.2 Watchdog Timer

⚠ WATCHDOG_TIMEOUT_MS = 2000ms config.h'da tanımlı ancak kodun hiçbir yerinde watchdog_enable(), watchdog_update() çağrıları yapılmıyor! Watchdog tamamen işlevsiz. RP2350'nin hardware watchdog'u sadece bootloader aşamasında çalışıyor olabilir. Bu kritik bir güvenlik açığı — uçuş döngüsü donduğunda sistem otomatik reset atmayacak.

7.3 Stack Overflow Koruması

⚠ FreeRTOS görev stack'leri 2048 word. Telemetry task'ı MAVLink + snprintf + blackbox yük ile bu limitin üzerine çıkabilir. vApplicationStackOverflowHook() tanımlı değil — stack taşması sessizce heap'i bozabilir veya hard fault oluşturabilir. configCHECK_FOR_STACK_OVERFLOW = 2 ve hook implementasyonu eklenmelidir.

7.4 Heap Yönetimi

BootLogger, setup() içinde hardcode edilmiş 182000 byte heap değerini raporluyor. Bu değer dinamik olarak xPortGetFreeHeapSize() ile alınmalıdır. Hardcode değer yanıltıcı olabilir.

```
// Mevcut (yanlış):
BootLogger::printHealthReport(..., 182000);
// Doğru:
BootLogger::printHealthReport(..., xPortGetFreeHeapSize());
```

7.5 Arm/Disarm Güvenliği

ARM şartı: throttle < 1050 && rudder > 1900, 1 saniye sürdürülmesi gerekiyor. Bu fiziksel stick kombinasyonu kasıtsız arm'ı önler.

DİSARM şartı: throttle < 1050 && rudder < 1100, 1 saniye. Güvenli.

Değerlendirme

Arm/Disarm mekanizması, standart RC transmitter protokollerinde kullanılan throttle-low + rudder-right/left kombinasyonunu taklit ediyor. 1 saniyelik hold zorunluluğu kazaları önler. ✓

7.6 Safe Mode / Brownout

⚠ Brownout detection ve safe mode mekanizması yok. Güç kaybında veya düşük voltajda davranış tanımsız. RP2350'nin hardware brownout detector'ı mevcut ama yazılımda kullanılmıyor. En azından ADC ile VBUS monitoring eklenmelidir.

BÖLÜM 8 — Haberleşme Analizi

8.1 MAVLink v2 Implementasyonu

MavlinkHandler.cpp, mavlink_parse_char() ile byte-by-byte parsing ve mavlink_msg_*_pack() ile message generation yapmaktadır. PIO UART üzerinden ESP32-CAM'e gönderilmektedir.

MAVLink Mesajı	Frekans	Yön	Durum
HEARTBEAT	1 Hz	FC → GCS	✓ Aktif
ATTITUDE	10 Hz	FC → GCS	✓ Aktif
RC_CHANNELS	5 Hz	FC → GCS	✓ Aktif
SYS_STATUS	2 Hz	FC → GCS	✓ Aktif (kısmi)
PARAM_REQUEST_LIST	N/A	GCS → FC	✓ Aktif
PARAM_SET	N/A	GCS → FC	✓ Aktif
RC_CHANNELS_OVERRIDE	N/A	GCS → FC	△ Parse var, işlenmiyor
COMMAND_LONG	N/A	GCS → FC	✗ Yok
MISSION_*	N/A	GCS → FC	✗ Yok (planned)
GPS_RAW_INT	N/A	FC → GCS	✗ Yok (GPS planned)

8.2 SYS_STATUS Mesajı Analizi

△ sendSysStatus() fonksiyonu batarya voltajı, akım ve CPU yük değerlerini 0/-1 gönderiyor. GCS'te bu değerlerin 'bilinmiyor' olarak görünmesi yerine -1 gönderimi bazı GCS yazılımlarında hata göstergesi olarak yorumlanabilir. Gerçek değerler veya UINT16_MAX (bilinmiyor sentinel) kullanılmalıdır.

8.3 Blackbox Logger

Blackbox, CSV formatında PIO UART üzerinden ESP32-CAM'e veri akışı yapıyor. Format: BB,timestamp,roll,pitch,yaw,gx,gy,gz,thr,ail,ele,rud,failsafe

50Hz TelemetryTask içinde çağrılıyor. snprintf kullanımı stack'e maliyet ekler ama kabul edilebilir.

Öneri

Blackbox verisi ESP32-CAM SD'ye yazılıyor — bu yaklaşım Pico flash'ı koruma altına alıyor. PX4'ün uLogger ve Betaflight'ın Blackbox protokolü ile kıyaslandığında format basit ama işlevsel. v0.4'te binary format (CSV yerine) bant genişliğini 3-4x azaltır.

8.4 Parametre Sistemi

ParamManager, MAVLink PARAM protokolü ile 6 PID parametresini GCS üzerinden ayarlanabilir yapıyor. MAVLINK_PARAMS_ENABLED makrosu ile koşullu derleniyor (varsayılan: devre dışı).

⚠ Parametreler SRAM'de tutuluyor — reset sonrası kaybolur. Persistent depolama (flash veya EEPROM emulation) eklenmesi gerekiyor. Pico'da littlefs veya RP2350 flash API ile çözülebilir.

BÖLÜM 9 — Pico 2 Donanım Analizi

9.1 RP2350 Özelliklerinin Kullanımı

RP2350 Özelliği	Kullanım	Etkinlik
Dual-Core ARM Cortex-M33	Core0: FreeRTOS, Core1: PID döngüsü	✓ Optimal
PIO State Machines	4x PWM servo + 2x UART = 6 SM	✓ Mükemmel
DMA Controller	MPU6050 I2C okuma	Δ Kısmi (TX hala blocking)
Hardware FPU	Madgwick float ops + PID hesapları	✓ Aktif
SRAM (264KB)	FreeRTOS heap + uçuş verileri	✓ Yeterli
Flash XIP (4MB)	Firmware + __not_in_flash_func	✓ Kritik kod RAM'de
USB (USBF)	Serial debug (Arduino USB CDC)	✓ Aktif
Hardware Watchdog	Tanımlı ama aktif edilmemiş	✗ Kritik eksik
ADC (12-bit)	Batarya monitoring yok	✗ Kullanılmıyor
SPI	Kullanılmıyor	Δ SD kart potansiyeli
Hardware Timer	pico/time.h ile sleep_us()	✓ Aktif
Hazard Counter	RP2350 güvenlik özelliği	✗ Kullanılmıyor

9.2 __not_in_flash_func() Kullanımı

Kritik fonksiyonlar __not_in_flash_func() ile işaretlenmiş:

- SensorFusion::update() ve updateIMU() — her tick çağrılıyor
- SensorFusion::computeAngles() — euler dönüşümü
- SensorManager::_mpu_parse() — DMA buffer parsing
- RingBuffer::push() ve pop() — core0/core1 iletişimi
- PID::compute() — PID hesabı

Bu optimizasyon, flash erişim wait state'lerini (XIP cache miss ~3-5 cycle) ortadan kaldırır. 500Hz döngüde gerçek performans artışı sağlar.

9.3 Overclock / Clock Yapılandırması

platformio.ini'de varsayılan 133MHz kullanılıyor. RP2350 resmi 150MHz, güvenli overclock 200-240MHz'e kadar çıkabilir. Madgwick filter ve PID için daha yüksek saat faydalı olabilir, ancak mevcut WCET ile gerek yok.

9.4 Bellek Haritası (Tahmini)

Bölge	Boyut	Kullanım
Flash (XIP)	4MB	Firmware ~200KB, libdeps ~500KB
SRAM0	128KB	FreeRTOS heap, task stacks
SRAM1	128KB	Uçuş verisi, DMA buffer, sensor buf
Scratch X/Y	8KB	__not_in_flash kod (RP2040) — RP2350'de farklı
SRAM tahmini kullanım	~180KB	xPortGetFreeHeapSize() ile doğrulanmalı

BÖLÜM 10 — PX4 / ArduPilot Karşılaştırması

10.1 Özellik Karşılaştırma Tablosu

Özellik	AeroPico v0.2	PX4 v1.15	ArduPilot 4.5
HAL Katmanı	△	✓	✓
FreeRTOS Scheduler	✓	✓	✓
Dual-Core Kullanımı	✓	△	△
PIO / Özel Donanım	✓	✗	✗
Cascaded PID	✓	✓	✓
EKF / EKF2	✗	✓	✓
Sensor Fusion (9DOF)	✓	✓	✓
Feed-Forward	✗	✓	✓
Uçuş Modları	✗	✓	✓
Mission / Waypoint	✗	✓	✓
GPS Navigasyon	✗	✓	✓
MAVLink	✓	✓	✓
Param Sistemi	△	✓	✓
Kalıcı Param Depolama	✗	✓	✓
GCS Tam Uyum	△	✓	✓
Blackbox Logging	✓	✓	✓
Watchdog	✗	✓	✓
Stack Guard	✗	✓	✓
Batarya Monitoring	✗	✓	✓
Sensor Kalibrasyon	✗	✓	✓
Hard/Soft Iron Comp.	✗	✓	✓
TECS (Enerji kontrolü)	✗	✓	✓
L1 Nav Controller	✗	✓	✓
Multi-platform	✗	✓	✓
Kod Okunabilirliği	✓	△	△
Başlangıç Kolaylığı	✓	✗	✗
RP2040/2350 Özelleşme	✓	✗	✗

10.2 AeroPico'nun Üstün Olduğu Alanlar

- RP2350'ye özgü PIO ve DMA optimizasyonu — PX4/ArduPilot generic HAL bu kadar düşük seviyeye inmiyor.
- Kod okunabilirliği ve eğitim değeri — 2200 satır, her modülü anlaşılır ve incelenebilir.
- Çift çekirdek kullanımı — PX4/ArduPilot tek çekirdek üzerinde scheduler kullanır.
- Minimum bağımlılık — Adafruit ve SBUS hariç external library az.
- Başlangıç eşiği düşük — PlatformIO + Pico, profesyonel SDK kurulumu gerektirmiyor.

10.3 Geliştirilmesi Gereken Alanlar

- EKF2 entegrasyonu: GPS + barometer + IMU füzyonu için Extended Kalman Filter.
- Uçuş modu mimarisi: MANUAL, FBWA, CRUISE, AUTO — state machine ile yönetim.
- HAL genelleştirmesi: Farklı sensör ve platform desteği için abstraction layer.
- Kalıcı parametre depolama: Flash API veya littlefs entegrasyonu.
- Sensor kalibrasyon rutini: Boot'ta gyro/accel bias hesaplama.

BÖLÜM 11 — Kod Denetimi (Bug Hunt)

11.1 Kritik Hatalar

BUG-001: Watchdog Hiç Başlatılmıyor

Konum: `main.cpp`, `config.h`

WATCHDOG_TIMEOUT_MS tanımlı ama `watchdog_enable(WATCHDOG_TIMEOUT_MS, true)` ve döngüde `watchdog_update()` çağrısı yok. Sistem donduğunda otomatik reset atmaz.

- Dosya: `main.cpp` `setup()` sonu ve `SystemTimer::core1_entry()` içi
- Çözüm: `watchdog.h` include + `watchdog_enable()` + periyodik `watchdog_update()`

BUG-002: `updateIMU()` Sadece Gyro Entegrasyonu

Konum: `SensorFusion.cpp:updateIMU()`

6-DOF Madgwick versiyonunda accelerometer gradient correction eksik. Accelerometer verisi normalizasyon dışında kullanılmıyor. Sabit ve düz uçuşta kabul edilebilir ama ivmelenme/frenleme sırasında pitch hatası birikir.

BUG-003: Heap Değeri Hardcode

Konum: `main.cpp:setup()` ve `taskTelemetry()`

`printHealthReport()` çağrılarında 182000 sabit değeri. Gerçek heap kullanımını yansıtmıyor. `xPortGetFreeHeapSize()` ile dinamik okunmalı.

11.2 Orta Seviye Sorunlar

WARN-001: `_mpu_start_dma_read()` Blocking Write

Konum: `Sensors.cpp:_mpu_start_dma_read()`

`i2c_write_blocking()` çağrısı register adresi göndermek için CPU'yu bloklıyor. Tam non-blocking için I2C TX de DMA ile yapılmalı.

WARN-002: `Wire.h` Blocking I2C (Mag + Baro)

Konum: `Sensors.cpp:update()`

`mag.getEvent()` ve `bmp.getEvent()` Arduino Wire kullanıyor. Wire blocking I2C yapıyor, bu da `SensorTask`'ı en fazla ~5ms geciktirebilir. Adafruit BusIO DMA desteği eklenmeli veya native SDK ile yeniden yazılmalı.

WARN-003: `Mixer.h` Kullanılmıyor

Konum: `src/core/Mixer.h/.cpp`

Servo kütüphanesi tabanlı Mixer sınıfı hiçbir yerde include edilmiyor. Dead code — silinmeli veya birleştirilmeli. Karışıklık ve bakım yükü oluşturuş.

WARN-004: `IMU.cpp` Kullanılmıyor

Konum: `src/drivers/IMU.h/.cpp`

IMUManager sınıfı legacy complementary filter içeriyor. SensorFusion.cpp Madgwick ile replace edilmiş ama IMU.cpp silinmemiş. Dead code.

WARN-005: PID Anti-Windup Yetersiz

Konum: PID.cpp:compute()

integral = constrain(integral, -100, 100) output satureasyonunu ele almıyor. Output sınırına ulaşıldığında integral büyümeye devam edebilir. Back-calculation anti-windup gerekli.

WARN-006: Tight Coupling — MavlinkHandler → FlightManager

Konum: MavlinkHandler.cpp, MavlinkHandler.h

MavlinkHandler, FlightManager'a extern referans alıyor (flightManager.getLatestData()). Bu katmanlar arası sıkı bağlantı, unit test yazmayı ve bağımsız gelişimi zorlaştırıyor.

WARN-007: snprintf Telemetry Task Stack Baskısı

Konum: Blackbox.cpp:log()

snprintf(buf, 128, ...) 128 byte stack frame ekliyor. TelemetryTask'ın 2048 word stack'i yeterli ama stack kullanımı izlenmiyor.

11.3 Düşük Öncelikli Uyarılar

- WARN-008: getLatestData() her çağrıda FlightData kopyası oluşturuyor — FlightManager'ın birden fazla get*() metodu her biri _getLatest() çağırıyor. Biri yeterli.
- WARN-009: Serial.println() FlightManager::updateArmDisarm() içinde — arm/disarm mesajları uçuş döngüsü içinde basılıyor, gecikme riski var.
- WARN-010: RC_CHANNELS_OVERRIDE MAVLink mesajı parse ediliyor ama FlightManager'a iletilmiyor (TODO yorumu). Bu eksik implementasyon GCS kontrolünü engelliyor.
- WARN-011: ParamManager değişiklik sonrası PID nesnelere uygulanmıyor. Parametre değişirse PID nesneleri eski katsayılarla çalışmaya devam eder.
- WARN-012: types.h'daki RC_Values struct hiçbir yerde kullanılmıyor — dead code.
- WARN-013: Serial.h ve Serial.cpp boş dosyalar — placeholder kalmış.

BÖLÜM 12 — Geliştirme Yol Haritası

12.1 v0.3 — Güvenlik ve Stabilité (Kısa Vadeli)

Tahmini süre: 2-4 hafta. Bu versiyon mevcut özelliklerin güvenilirliğini garanti altına alır.

1. Watchdog aktiveştirme: `watchdog_enable()` + periyodik `watchdog_update()` — kritik öncelik.
2. FreeRTOS stack overflow hook: `configCHECK_FOR_STACK_OVERFLOW = 2`, `vApplicationStackOverflowHook()` implementasyonu.
3. `updateIMU()` 6-DOF Madgwick düzeltmesi: Accelerometer gradient correction eklenmesi.
4. `xPortGetFreeHeapSize()` ile dinamik heap monitoring.
5. Dead code temizliği: `Mixer.h`, `IMU.cpp`, `types.h` `RC_Values`, `Serial.h`.
6. PID back-calculation anti-windup implementasyonu.
7. Gyro/Accel boot kalibrasyonu: İlk 100 sample ortalama ile bias hesaplama.

12.2 v0.4 — Özellik Genişletme (Orta Vadeli)

Tahmini süre: 1-2 ay. Temel uçuş yetenekleri genişletilir.

8. Uçuş modu state machine: MANUAL, FBWA (Fly-By-Wire-A), STABILIZE modları.
9. GPS NMEA parser entegrasyonu — UART1 GP8/GP9.
10. Barometrik irtifa tutma (altitude hold) — basit P kontrolcüsü.
11. Parametre kalıcı depolama: RP2350 flash API ile PID değerlerini kaydetme.
12. Mag hard-iron kalibrasyonu: Min/Max yöntemi ile offset hesaplama.
13. `RC_CHANNELS_OVERRIDE` MAVLink implementasyonu.
14. Binary Blackbox format (CSV yerine) — bant genişliği optimizasyonu.

12.3 v0.5 — Navigasyon (Uzun Vadeli)

Tahmini süre: 2-4 ay. Temel özerk uçuş yetenekleri.

15. EKF implementasyonu veya porting: GPS + Baro + IMU füzyonu.
16. L1 navigasyon kontrolcüsü: Waypoint takip algoritması.
17. TECS (Total Energy Control System): Hız ve irtifa eşgüdümlü kontrol.
18. AUTO uçuş modu: Basit waypoint navigasyonu.
19. Batarya voltaj monitoring: ADC + Kalibrasyon.
20. GCS tam uyum: Mission Planner / QGroundControl ile tam görsel telemetri.

12.4 v1.0 — Kararlı Sürüm

Tahmini süre: 4-6 ay. Güvenilir prototip ve eğitim platformu.

21. Kapsamlı birim testleri (host-side): PID, SensorFusion, Mixer testleri.
22. Hardware-in-the-loop (HIL) simülasyon desteği.
23. Kapsamlı dokümantasyon: Doxygen API docs.
24. SITL (Software in the Loop) desteği.
25. Stabil MAVLink dialect ile QGroundControl tam uyumu.

12.5 v2.0 — Platform Geniřlemesi

- 26. HAL (Hardware Abstraction Layer) geliřtirmesi: Farklı Pico/RP2350 board desteęi.
- 27. Quadrotor modu desteęi: Multirotor mixer.
- 28. ROS2 uNode entegrasyonu arařtırması.
- 29. Yapay zeka tabanlı PID autotuning.

EKLER

Ek A — Önceliklendirilmiş TODO Listesi

Öncelik	Görev	Dosya	Tahmini Süre
☐ KRİTİK	Watchdog aktifleştir	main.cpp	1 saat
☐ KRİTİK	updateIMU() gradient ekle	SensorFusion.cpp	2 saat
☐ KRİTİK	Stack overflow hook	main.cpp	1 saat
☐ YÜKSEK	Gyro bias kalibrasyonu	Sensors.cpp	3 saat
☐ YÜKSEK	PID anti-windup back-calc	PID.cpp	2 saat
☐ YÜKSEK	Heap monitoring dinamik	main.cpp	30 dk
☐ YÜKSEK	Dead code temizliği	Mixer.h, IMU.cpp	1 saat
☐ ORTA	RC_OVERRIDE implementasyonu	MavlinkHandler.cpp	3 saat
☐ ORTA	Param → PID runtime güncelleme	ParamManager.cpp	2 saat
☐ ORTA	Wire blocking → async	Sensors.cpp	4 saat
☐ DÜŞÜK	Binary blackbox format	Blackbox.cpp	4 saat
☐ DÜŞÜK	Mag hard-iron kalibrasyon	Sensors.cpp	3 saat

Ek B — Risk Matrisi

Risk	Olasılık	Etki	Azaltma Stratejisi
Watchdog devre dışı → Uçuş döngüsü donması	Orta	Kritik	v0.3'te watchdog_enable()
updateIMU() drift → Yanlış attitude	Yüksek	Yüksek	6-DOF Madgwick düzeltmesi
Stack overflow → Bellek bozulması	Düşük	Kritik	Overflow hook + stack artır
Wire blocking → Döngü zamanlaması bozulması	Orta	Yüksek	Native I2C DMA veya async
Sensor kalibrasyonsuz → Hatalı açılar	Yüksek	Yüksek	Boot kalibrasyonu

MAVLink tight coupling → Geliştirme zorluğu	Yüksek	Düşük	Dependency injection
Param reset sonrası kaybolma → Tuning kaybı	Yüksek	Orta	Flash kalıcı depolama

Ek C — Modül Puan Özeti

Kriter	Puan	Not
Mimari Genel	8/10	Katmanlı yapı, çift çekirdek izolasyonu
PIO / Donanım Kullanımı	9/10	RP2350 tam kapasitede kullanılıyor
DMA Sensör Okuma	7/10	TX hala blocking, kısmi DMA
Madgwick (9-DOF)	8/10	Implementasyon doğru, temp comp. ekstra
Madgwick (6-DOF)	4/10	Accel gradient eksik, drift riski yüksek
Cascaded PID	7/10	Doğru mimari, anti-windup zayıf
FixedWingMixer	8/10	Gain + trim + constrain, temiz
RXManager / SBUS	9/10	Çift failsafe, timeout, channel map
MAVLink Handler	7/10	Parse + send çalışıyor, override eksik
Blackbox	7/10	Basit ama işlevsel, binary format yok
ParamManager	6/10	Protokol tamam, PID uygulama ve kalıcılık yok
Güvenlik / Watchdog	3/10	Watchdog aktif değil, kritik eksik
Kod Kalitesi / Okunurluk	8/10	Yorum zengin, modüler, anlaşılır
Test / Validation	2/10	Test altyapısı yok